Practical Lab:
Access Control Models

Omar Ait Said | Information Security | UM6P | November 2026

In this lab I studied the four main access control models: **DAC** (ownership), **MAC** (central policy), **RBAC** (roles) and **ABAC** (attributes and conditions). I tested each one on Linux with real users, groups and permissions.

I don't have Linux on my laptop, so I installed Ubuntu with **WSL 2** (**`wsl --install`**) and did the whole lab inside it, working in my Linux home folder (~) because Windows drives (**`/mnt/c`**) ignore Linux permissions.

```
wsl --install          # PowerShell (Administrator)
```

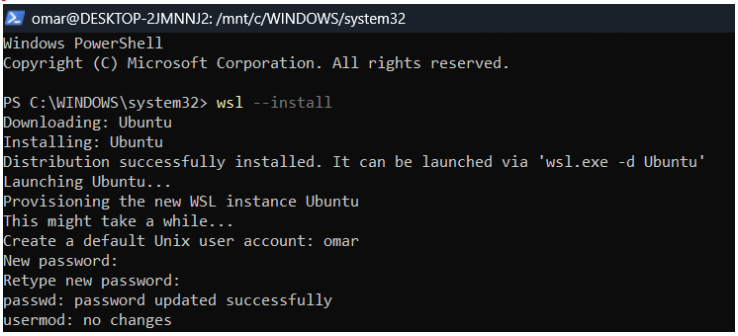


Figure 1 – Installing Ubuntu with WSL

```
whoami
id
cat /etc/os-release | head -n 4
uname -r
```

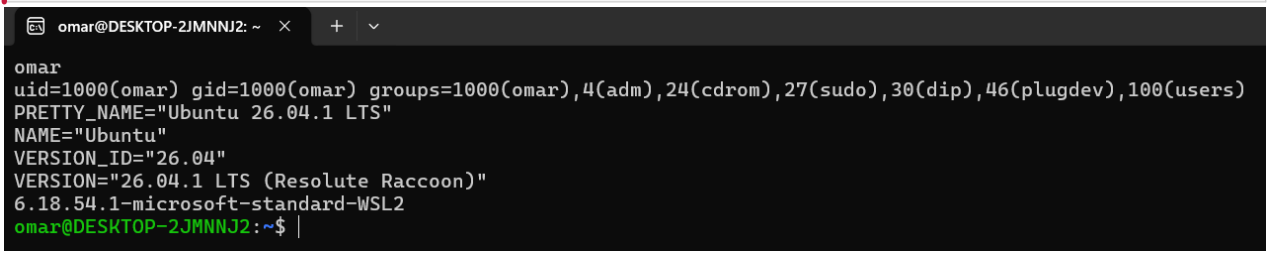


Figure 2 – Environment: user omar (in sudo), Ubuntu 26.04.1 LTS, WSL2 kernel

1 – Discretionary Access Control (DAC)

In DAC the owner of a file decides who can access it. Each file has an owner, a group and three permission triplets (owner / group / others), with r = 4, w = 2, x = 1.

Guided experiment

After **`touch report.txt`** the file is **`-rw-r--r--`** (644): owner read/write, group and others read only, because the default umask 022 removes write. After **`chmod 740`** it becomes **`-rwxr-----`**: 7 = 4+2+1 = rwx (owner), 4 = r-- (group), 0 = --- (others).

```
mkdir ~/DAC_Lab
cd ~/DAC_Lab
touch report.txt
ls -l report.txt
```

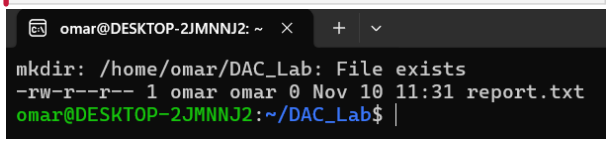


Figure 3 – Default permissions (644)

```
chmod 740 report.txt
ls -l report.txt
```

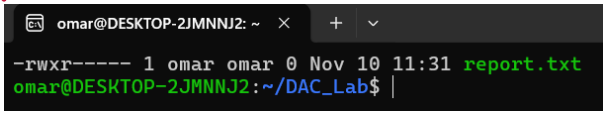


Figure 4 – After chmod 740

Then **`sudo groupadd accounting`** and **`sudo chown root:accounting report.txt`** give **`-rwxr----- root accounting`**. Now I am neither the owner nor in *accounting*, so I fall in **others** (---) and I can't read the file anymore. Only the owner (or root) can change this, which is the idea of DAC.

Challenge 1 – Confidential Documents

1. Create the directory and a document. I created **`/company/finance`** and **`budget_2026.txt`** with sudo. By default they are owned by root with 755/644, so everyone can read them: the policy is not respected yet.

```
sudo mkdir -p /company/finance
echo "Q3 budget: 1,200,000 MAD" | sudo tee /company/finance/budget_2026.txt
ls -ld /company/finance
ls -l /company/finance
```

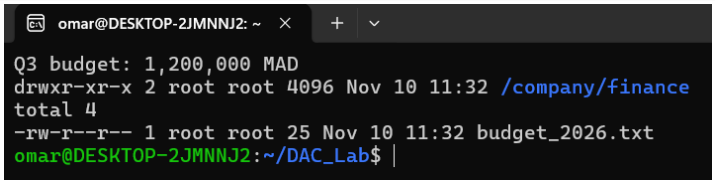


Figure 5 – Directory and document created (root, 755 / 644)

2. Create the accounting group. The group already existed from the guided part. To test the policy I created two users: **`accountant`** (added to *accounting*) and **`outsider`** (no group).

```
sudo useradd -m -s /bin/bash accountant
sudo usermod -aG accounting accountant
sudo useradd -m -s /bin/bash outsider
getent group accounting
id accountant
id outsider
```

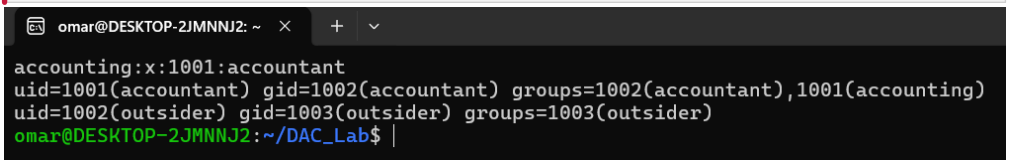


Figure 6 – accountant is in accounting, outsider is not

3–4. Ownership and permissions. **`chown -R omar:accounting /company/finance`**, then **`chmod 750`** on the directory and **`chmod 640`** on the file:

Object	Mode	Owner (omar)	Group (accounting)	Others
/company/finance	750	rwx: list, enter, create	r-x: list and enter	---
budget_2026.txt	640	rw-: read and modify	r--: read only	---

The group needs **x** on the directory to reach the files and **r** to list them. The file needs no x because it is a document, not a program.

```
sudo chown -R omar:accounting /company/finance
sudo chmod 750 /company/finance
sudo chmod 640 /company/finance/budget_2026.txt
ls -ld /company/finance
ls -l /company/finance
```

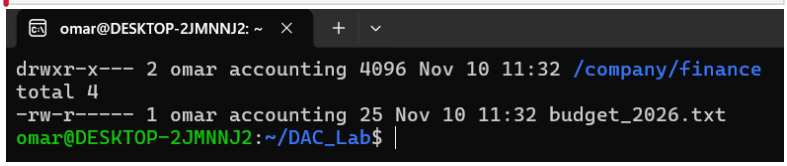


Figure 7 – Final configuration: drwxr-x--- and -rw-r-----, omar:accounting

5. Verify. I tested each user with **`sudo -u`**: the owner reads and writes (OWNER WRITE OK), *accountant* reads but the write is **denied**, and *outsider* cannot even open the directory. The policy is respected.

```
cat /company/finance/budget_2026.txt
echo "Q4 forecast added" >> /company/finance/budget_2026.txt && echo "OWNER WRITE OK"
sudo -u accountant cat /company/finance/budget_2026.txt
sudo -u accountant bash -c 'echo hack >> /company/finance/budget_2026.txt'
sudo -u outsider ls /company/finance
sudo -u outsider cat /company/finance/budget_2026.txt
```

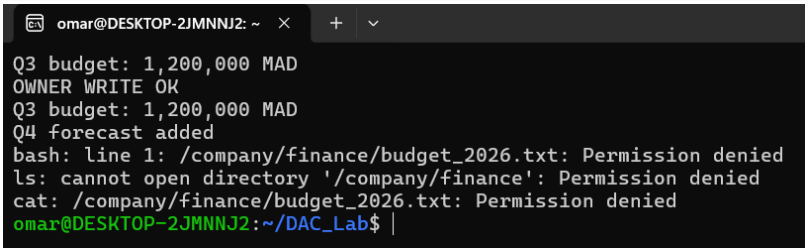


Figure 8 – Owner: read + write · accountant: read only · outsider: no access

6. Directory 750 but file 600: can an accounting member read it? No. The directory gives the group r-x, so the member can enter it and see the file name. But reading the content depends on the file's own permissions, and

600 gives the group ---. Both the directory and the file must allow the access. I tested it, then put the file back to 640.

```
sudo chmod 600 /company/finance/budget_2026.txt
ls -l /company/finance/budget_2026.txt
sudo -u accountant ls -l /company/finance
sudo -u accountant cat /company/finance/budget_2026.txt
sudo chmod 640 /company/finance/budget_2026.txt     # restore
```

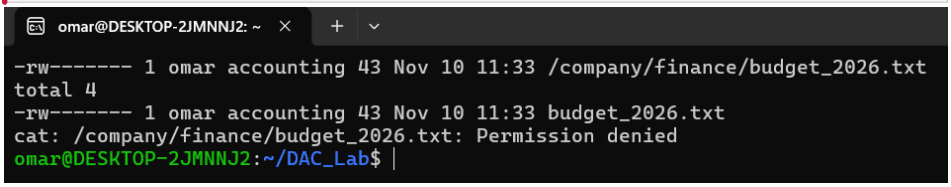


Figure 9 – With 600, the group can list the directory but cat is denied

2 – Mandatory Access Control (MAC)

In MAC a central security policy decides, and the owner cannot change it. Bell-LaPadula protects confidentiality with **no read up** and **no write down**.

SELinux exploration. `sestatus` was not installed. After installing `policycoreutils` it shows **SELinux status: disabled**, and `ls -Z` shows **?** instead of a security context.

This is normal: Ubuntu uses AppArmor instead of SELinux, and the WSL kernel does not enforce SELinux. On Fedora or Rocky Linux, `sestatus` would say *enforcing* and each file would have a label like `user:role:type:level`, checked by the kernel after DAC.

```
sestatus
sudo apt install -y policycoreutils
sestatus
ls -Z ~/DAC_Lab
```

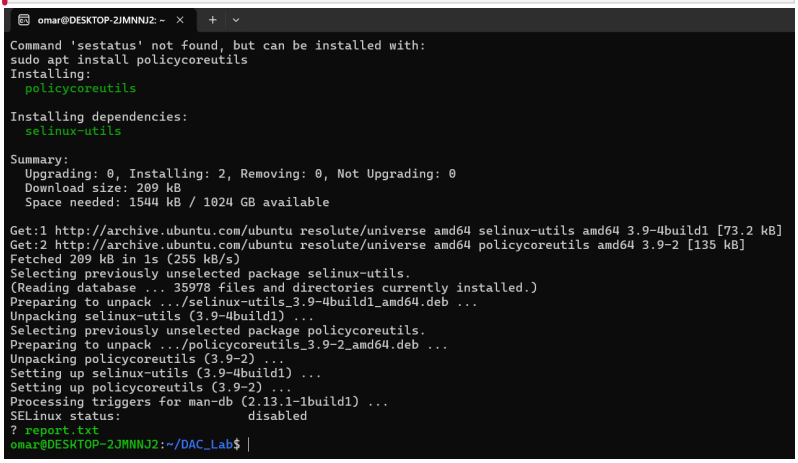


Figure 10 – sestatus and ls -Z on WSL

Challenge 2 – Military Information System

1. Access matrix. Order: Public < Confidential < Secret. A user reads at or below his level and writes at or above it.

User (clearance)	Document_A (Secret)	Document_B (Confidential)	Document_C (Public)
Alice (Secret)	Read, Write	Read	Read
Bob (Confidential)	Write	Read, Write	Read
Charlie (Public)	Write	Write	Read, Write

For reading: Alice reads A, B and C; Bob reads B and C; Charlie reads only C.

2. Bob reads Document_A? Denied. Bob is Confidential and A is Secret: it is a **read up**.

3. Alice writes into Document_C? Denied. Alice is Secret and C is Public: it is a **write down**. She could copy secret data into a public document.

4. How Bell-LaPadula prevents leakage. No read up stops a user from reading data above his clearance, and no write down stops him (or a malware running with his clearance) from copying high data into a lower document. So information can only go up, never down.

5. Enough clearance, but the owner denies with Linux permissions? Access is **not** granted. In a system like SELinux the request must pass **both** checks: DAC first, then MAC. MAC can only restrict more, it never allows what DAC refuses. The difference: in DAC the **owner** decides and can change permissions; in MAC the **administrator's central policy** decides and users cannot override it.

3 – Role-Based Access Control (RBAC)

In RBAC permissions are given to roles, and users get roles. I used Linux groups as roles.

Challenge 3 – Enterprise Access Management

1. Groups and users. I created the groups *administrators*, *developers*, *viewers*, the users alice, bob and charlie with **`adduser`**, then assigned the roles with **`usermod -aG`**.

```
sudo groupadd administrators
sudo groupadd developers
sudo groupadd viewers
sudo adduser alice      # same for bob, charlie
```

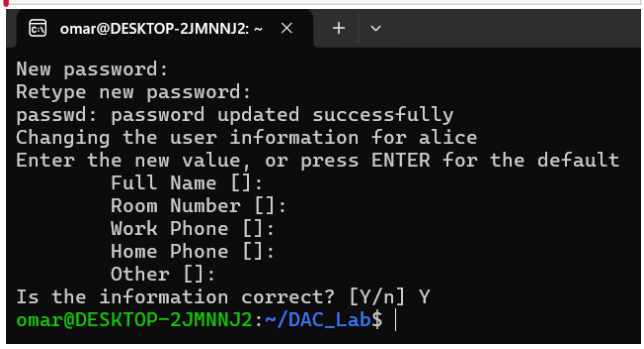


Figure 11 – Creating user alice

```
sudo usermod -aG developers alice
sudo usermod -aG viewers bob
sudo usermod -aG administrators charlie
groups alice
groups bob
groups charlie
```

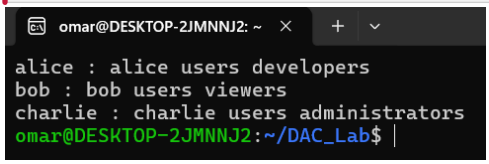


Figure 12 – alice: developers, bob: viewers, charlie: administrators

2. Configure the directories. As in the lab: **`/company/source`** is root:developers 770 and **`/company/reports`** is root:viewers 750. I added one file in each to test.

```
sudo mkdir -p /company/source /company/reports
sudo chown root:developers /company/source
sudo chmod 770 /company/source
sudo chown root:viewers /company/reports
sudo chmod 750 /company/reports
echo 'int main(){ return 0; }' | sudo tee /company/source/app.c
echo 'Q3 report: revenue up 12%' | sudo tee /company/reports/q3_report.txt
ls -ld /company/source /company/reports
```

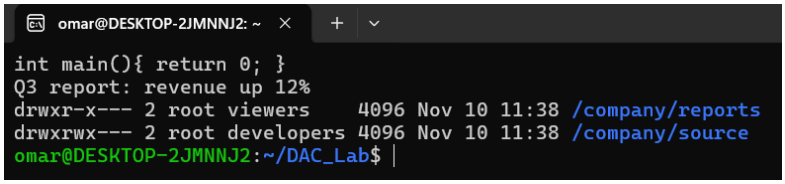


Figure 13 – Lab configuration of the two directories

3. Does it satisfy the policy? No. A user who is not in the directory's group falls in **others**, which has --- on both directories:

Role (user)	Source: required → actual	Reports: required → actual	Result
Developer (alice)	Read/Write → rwx (group)	Read → --- (others)	Reports missing
Viewer (bob)	No access → --- (others)	Read → r-x (group)	OK
Administrator (charlie)	Full → --- (others)	Full → --- (others)	Both missing

```
sudo -u alice touch /company/source/feature.c && echo "alice writes source: OK"
sudo -u alice ls /company/reports
sudo -u bob ls /company/reports
sudo -u bob ls /company/source
sudo -u charlie ls /company/source
sudo -u charlie ls /company/reports
```

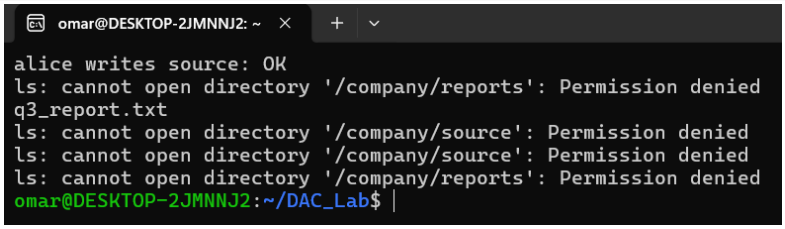


Figure 14 – alice is denied on reports, charlie is denied on both directories

4. Correct the missing permissions. Basic permissions allow only one group per file, but here three roles need different rights on the same directory. So I used **ACLs**:

```
sudo apt install -y acl
sudo setfacl -R -m g:administrators:rwX /company/source /company/reports
sudo setfacl -R -m g:developers:rwX /company/source
sudo setfacl -R -m g:developers:rX /company/reports
sudo setfacl -R -d -m g:administrators:rwX,g:developers:rwX /company/source
sudo setfacl -R -d -m g:administrators:rwX,g:developers:rX /company/reports
```

`-R` applies the rule to existing files, `-d` sets the default for new files, and `X` gives execute only to directories.

`getfacl` shows the new entries: administrators rwx on both, developers rwx on source and r-x on reports. `ls -ld` now shows a **+**. With an ACL, the middle triplet shown by `ls` is the **mask**, not the group's permissions, which is why reports shows rwx while viewers still have r-x.

Adding charlie to *viewers* instead would not work: he needs write on reports, and giving it to the viewers group would break the viewer rule.

```
getfacl /company/source
getfacl /company/reports
ls -ld /company/source \
       /company/reports
```

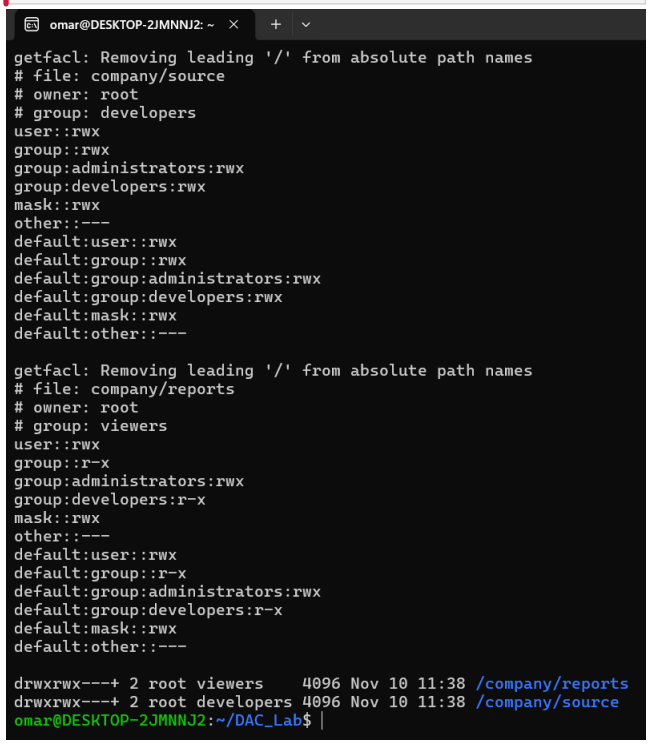


Figure 15 – getfacl after the correction

```
sudo -u alice cat /company/reports/q3_report.txt
sudo -u alice touch /company/reports/alice.txt
sudo -u bob cat /company/reports/q3_report.txt
sudo -u bob touch /company/reports/bob.txt
sudo -u bob ls /company/source
sudo -u charlie touch /company/source/admin.c && echo "charlie writes source: OK"
sudo -u charlie bash -c 'echo "approved" >> /company/reports/q3_report.txt' && echo "charlie writes
reports: OK"
```

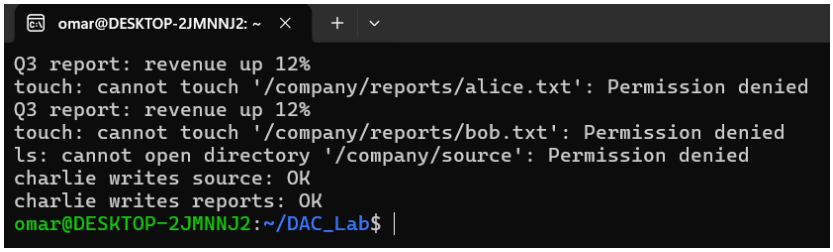


Figure 16 – After correction: alice reads reports (no write), bob read-only, charlie writes both

Now every rule of the policy is respected: developer read/write source + read reports, viewer read reports only, administrator full access to both.

4 – Attribute-Based Access Control (ABAC)

In ABAC the decision uses attributes of the subject, the object, the action and the environment. Here all three conditions must be true (AND).

Challenge 4 – Banking Access Policy

1–2. Decisions and attribute responsible.

User	Department	Clearance	Time	Decision	Responsible attribute
Alice	Finance = Finance	Secret ≥ Secret	10:00 in	**Granted**	–
Bob	HR ≠ Finance	Conf. ≥ Conf.	11:00 in	**Denied**	Department (subject vs object)
Charlie	Finance = Finance	Public < Secret	14:00 in	**Denied**	Clearance (subject vs object)
David	Finance = Finance	Secret ≥ Secret	22:00 out	**Denied**	Time (environment)

3. Alice requests an HR document. Denied: her department (Finance) does not match HR. Her Secret clearance doesn't help because one false condition is enough to deny.

4. David at 17:00. Yes, it becomes **granted**: department and clearance already matched, and 17:00 is inside 08:00–18:00. Same user, same document, different decision because of the context.

5. Why ABAC is more flexible than RBAC. RBAC only looks at the role, so each new condition needs a new role (role explosion). ABAC combines many attributes in one rule, including context like time or location, so it is much more fine-grained. The cost is that policies are harder to write and audit.

To check my answers I wrote a small Python script that implements the policy.

```
cat > ~/abac.py << 'EOF'   # script typed in, ended with EOF
cat ~/abac.py
```

```
omar@DESKTOP-2JMNNJ2: ~

LEVELS = {"Public": 0, "Confidential": 1, "Secret": 2}

def minutes(hhmm):
    h, m = map(int, hhmm.split(":"))
    return h * 60 + m

def evaluate(user, dept, clearance, doc_dept, doc_class, time):
    reasons = []
    if dept != doc_dept:
        reasons.append("department mismatch")
    if LEVELS[clearance] < LEVELS[doc_class]:
        reasons.append("insufficient clearance")
    if not (minutes("08:00") <= minutes(time) <= minutes("18:00")):
        reasons.append("outside 08:00-18:00")
    decision = "GRANT" if not reasons else "DENY"
    why = "" if not reasons else "  <- " + ", ".join(reasons)
    print(f"{user:8}{dept:9}{clearance:14}{doc_dept + '/' + doc_class:22}{time}  {decision}{why}")

print("Task 1 - lab requests")
evaluate("Alice", "Finance", "Secret", "Finance", "Secret", "10:00")
evaluate("Bob", "HR", "Confidential", "Finance", "Confidential", "11:00")
evaluate("Charlie", "Finance", "Public", "Finance", "Secret", "14:00")
evaluate("David", "Finance", "Secret", "Finance", "Secret", "22:00")
print("\nTask 3 - Alice requests an HR document")
evaluate("Alice", "Finance", "Secret", "HR", "Secret", "10:00")
print("\nTask 4 - David requests at 17:00")
evaluate("David", "Finance", "Secret", "Finance", "Secret", "17:00")
omar@DESKTOP-2JMNNJ2:~/DAC_Lab$
```

Figure 17 – abac.py: the three conditions of the policy

```
python3 ~/abac.py
```

```
omar@DESKTOP-2JMNNJ2: ~

Task 1 - lab requests
Alice   Finance  Secret        Finance/Secret        10:00  GRANT
Bob     HR       Confidential  Finance/Confidential  11:00  DENY  <- department mismatch
Charlie Finance  Public        Finance/Secret        14:00  DENY  <- insufficient clearance
David   Finance  Secret        Finance/Secret        22:00  DENY  <- outside 08:00-18:00

Task 3 - Alice requests an HR document
Alice   Finance  Secret        HR/Secret             10:00  DENY  <- department mismatch

Task 4 - David requests at 17:00
David   Finance  Secret        Finance/Secret        17:00  GRANT
omar@DESKTOP-2JMNNJ2:~/DAC_Lab$
```

Figure 18 – Output: same decisions as my table, including tasks 3 and 4

Conclusion

- **DAC:** simple and flexible, but security depends on each owner's choices.
- **MAC:** a central policy that owners cannot override; strong for confidentiality (Bell-LaPadula).
- **RBAC:** easy to manage through roles, but basic Linux groups were too coarse, so I needed ACLs.
- **ABAC:** the most flexible, it adds context like time, but policies become more complex.

In practice these models are combined: Linux checks DAC first, then MAC (SELinux/AppArmor), and organisations use roles and attributes on top.